Przepisywanie zapytań w silniku RSONPath

Projekt badawczy w ramach seminarium magisterskiego DeSeR

WIKTOR RUTECKI, Uniwersytet Warszawski, Polska
PIOTR PŁYWACZ, Uniwersytet Warszawski, Polska

Additional Key Words and Phrases: JSONPath, RSONPath, JSON, SIMD, przepisywanie zapytań, automaty skończone

1 WPROWADZENIE

Ze względu na uniwersalność i prostotę, format JSON jest standardem przechowywania i wymiany danych w internecie. JSONPath jest językiem zapytań, który pozwala przeszukiwać dokumenty JSON. Projekt RSONPath [RSO26] implementuje silnik wykonywania zapytań JSONPath. Jego wiodącą cechą jest wykorzystywanie instrukcji wektorowych procesora (SIMD) w celu umiejętnego „przewijania" fragmentów dokumentu, które nie są istotne dla wyniku zapytania.

Praca z konferencji ASPLOS '23 [GMP24] opisuje działanie silnika RSONPath i zawiera wyniki eksperymentów, które wskazują, że na czas wykonywania zapytania wpływa nie tylko rozmiar wejścia i wyniku zapytania, ale i sposób jego sformułowania. W tej pracy prezentujemy moduł systemu RSONPath, który wspiera użytkowników w przepisywaniu zapytań na równoważne w sensie wyniku, ale wydajniejsze w wykonaniu dzięki wykorzystaniu charakterystyki silnika.

Podstawy działania JSONPath i silnika RSONPath opisujemy w rozdziale 2. Architekturę optymalizatora do przepisywania zapytań JSONPath i jego komponenty opisujemy w rozdziale 3. Szczegóły implementacyjne opisane są w rozdziale 4. W rozdziale 5 znajduje się opis przeprowadzonych eksperymentów i ich wyniki. W ostatnim, 6 rozdziale zamieściliśmy podsumowanie, wnioski i plany na przyszłość związane z rozwojem tego projektu.

2 TŁO: SILNIK RSONPATH

2.1 JSONPath — język zapytań do dokumentów JSON

JSONPath [GNB24] to język zapytań dla dokumentów JSON. Interpretując dokument JSON jako drzewo, zapytanie definiuje wzorzec ścieżki jako ciąg etykiet w wierzchołkach drzewa. Wykonanie zapytania zwraca treść wszystkich poddokumentów, do których prowadzi ścieżka pasująca do zadanego wzorca.

Wzorzec ścieżki składa się z segmentów — każdy segment to selektor i etykieta. Zapytanie przetwarzane jest segment po segmencie. W każdym kroku obliczany jest zbiór wierzchołków, które pasują do aktualnego prefiksu zapytania. Dopasowując następny segment, dla każdego wierzchołka z poprzedniego kroku znajdowane są wierzchołki należące do jego poddrzewa, które pasują do tego selektora i etykiety w tym segmencie. Wynik zapytania to zbiór wierzchołków otrzymany po przetworzeniu ostatniego segmentu.

W pierwszym segmencie wzorca zawsze występuje selektor korzenia $ — inicjalizuje zbiór wierzchołków na singleton korzenia drzewa JSON. W kolejnych krokach wyróżniamy dwa typy selektorów:

- `.label` — selektor dziecka — dla każdego wierzchołka z poprzedniego zbioru akceptowany jest wierzchołek będący jego bezpośrednim dzieckiem, jeśli ma odpowiednią etykietę *label*.
- `..label` — selektor potomka — dla każdego wierzchołka z poprzedniego zbioru akceptowany jest wierzchołek będący jego potomkiem, jeśli ma odpowiednią etykietę *label*.

Authors' addresses: Wiktor Rutecki, , Uniwersytet Warszawski, Warszawa, Polska; Piotr Pływacz, , Uniwersytet Warszawski, Warszawa, Polska.

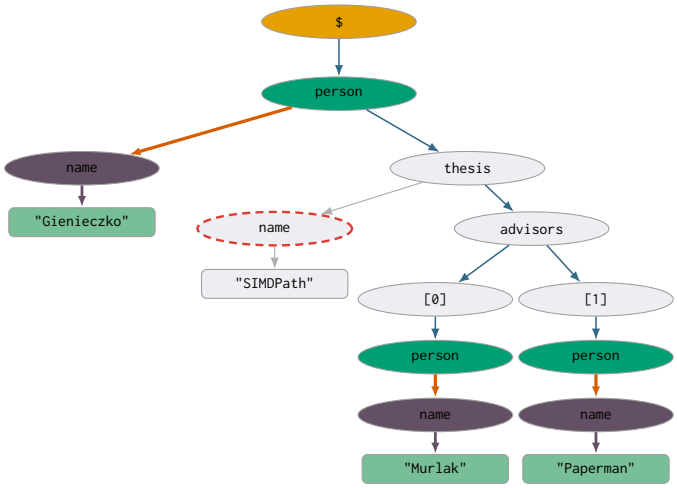


Fig. 1. Przykład wykonania zapytania `$..person.name` na drzewie JSON

Przykładowe zapytanie pokazujemy na obrazku 1. W pierwszym kroku zaznaczany jest korzeń drzewa. W drugim kroku zaznaczane są wszystkie potomki korzenia z etykietą `person`. W trzecim kroku zaznaczane są wszystkie dzieci wierzchołków z etykietą `person` z etykietą `name`.

2.2 Praktyczna implementacja wykonywania zapytań w silniku RSONPath

RSONPath implementuje wykonywanie zapytań JSONPath. Zapytanie zapisywane jest jako automat deterministyczny — każda napotkana etykieta na ścieżce od korzenia zmienia stan dopasowania ścieżki do wzorca z zapytania. Silnik przeszukuje drzewo dokumentu wgłąb, dla każdego wierzchołka utrzymuje informację o indeksie stanu automatu, w którym się znajduje w tej chwili (intuicyjnie odpowiada to mniej więcej długości prefiksu zapytania, który już został dopasowany). Przy przejściu do dziecka jest odwzorowane poprzez przejście w automacie zapytania po krawędzi z odpowiednią etykietą. podczas gdy powrót w górę drzewa, powoduje odtworzenie automatu do stanu zachowanego w rodzicu.

Taka implementacja pozwala na strumieniowe przetwarzanie dokumentu, ponieważ wystarcza stos przechowujący indeksy stanów, które właściwe dla wierzchołków na ścieżce z korzenia do obecnie odwiedzanego wierzchołka, a także pamiętanie automatu zapytania (o niewielkim rozmiarze względem całości dokumentu).

Stan automatu zmienia się jedynie, gdy napotka etykietę (przejście w dół drzewa) lub nawias zamykający (powrót do rodzica). Dzieli to znaki w strumieniu na dwie kategorie: interesujące z perspektywy dopasowywania zapytania i pozostałe znaki, które nie są aż tak istotne (np. wartości w liściach, białe znaki formatujące dokument wizualnie) W celu zwiększenia wydajności silnik wykorzystuje specjalne wektorowe instrukcje procesora (Single Instruction, Multiple Data, SIMD [Lam75]), które pozwalają wykonywać tą samą operację na kilku znakach w jednym cyklu procesora. Przy ich użyciu możemy znacząco przyspieszyć przetwarzanie strumienia znaków, „pomijając" nieistotne znaki. Mechanizm pomijania został zaimplementowany jako iterator, który z strumienia będącego dokumentem JSON zwraca kolejne pozycje, na których występują znaki strukturalne: `{`, `}`, `:`, `[`, `]`, `"`.

Logikę pomijania można rozszerzyć: iterator pozwala pomijać (w trybie SIMD) nie tylko do najbliższego znaku strukturalnego, ale również do najbliższego nawiasu zamykającego o określonej, wybranej głębokości — co efektywnie

ID	Zapytanie descendant-free	Przepisane zapytanie	# wyników
B1	`$.products.*.categoryPath.*.id`	`$..categoryPath..id`	697 440
B2	`$.products.*.videoChapters.*.chapter`	`$..videoChapters..chapter`	8 857
*B3	`$.products.*.videoChapters`	`$..videoChapters`	769
G2	`$.*.available_travel_modes`	`$..available_travel_modes`	90
W1	`$.items.*.bestMarketplacePrice.price`	`$..bestMarketplacePrice.price`	15 892
W2	`$.items.*.name`	`$..name`	272 499
Wi	`$.*.claims.P150.*.mainsnak.property`	`$..P150..mainsnak.property`	15 603

Tabela 1. Wyniki z pracy ASPLOS '23

pomija przetwarzanie całego poddrzewa, jeśli stan automatu wskazuje na to, że nie znajdziemy tam żadnego dopasowania. Przykładem efektywnego pomijania jest sytuacja, w której szukamy przejścia po selektorze potomka — konwencjonalnie JSONy nie pozwalają na powtarzanie się etykiet w jednym dokumencie, zatem po znalezieniu jednego poddokumentu z odpowiednią etykietą całą resztę tego dokumnetu możemy przewinąć.

Drugim znaczącym scenariuszem, w którym przewijamy dużą ilość dokumentu jest początek zapytania, jeśli zaczyna się od selektora potomka `..`. Wówczas możemy pomijać treść dokumentu aż do napotkania tej etykiety, gdziekolwiek w dokumencie się ona znajduje.

2.3 Wyniki z pracy ASPLOS '23 jako motywacja dla projektu

Praca „Supporting Descendants in SIMD-Accelerated JSONPath" [GMP24] stanowi punkt wyjścia naszego projektu. Autorzy, wprowadzając obsługę selektora potomka `..`, wykonali eksperymenty wydajnościowe silnika RSONPath dla zapytań benchmarkowych JSONPath. Każde zapytanie zostało przygotowane w dwóch wariantach: oryginalnym, bez wykorzystania selektora potomka (*descendant-free*) oraz w wersji z wykorzystaniem selektora potomka. Te drugie zostały przepisane ręcznie w taki sposób, aby zwracane przez nie wyniki na konkretnych dokumentach były identyczne jak w oryginale.

Zapytania i ich przepisane wersje przytaczamy w tabeli 1, wraz z wykresem z tamtej pracy, który obrazuje wyniki wydajnościowe. Istotny z naszej perspektywy jest fakt, że wykorzystanie selektora potomka znacząco zwiększa przepustowość systemu w wielu przypadkach. Takie zapytania lepiej wykorzystują SIMD, ponieważ stwarzają większe możliwości pomijania fragmentów dokumentu.

Nasz projekt opisywany w tym dokumencie miał za zadanie dalsze zbadanie tego tropu i stworzenie całego modułu-optymalizatora zapytań JSONPath świadomego specyfiki silnika RSONPath.

2.4 Wykonanie zapytań JSONPath jako teoria automatów

W tej pracy będziemy posługiwać się terminologią związaną z teorią języków i automatów do opisania działania stworzonych algorytmów. Alfabetem Σ będzie zbiór wszystkich etykiet, które występują w danym problemie (np. w dokumencie lub zapytaniu), wraz z specjalnym symbolem `[*]` oznaczającym etykietę elementu listy. Słowem nad alfabetem będzie ciąg etykiet, dla wygody przy zapisie słowa litery (etykiety) rozdzielamy kropkami.

Zapytanie JSONPath traktujemy jako wzorzec, do którego próbujemy dopasować ścieżki występujące w sprawdzanym dokumencie. Wzorzec definiuje zbiór ścieżek (potencjalnie nieskończony), które spełniają jego warunki. Jednocześnie jest to zbiór słów nad alfabetem Σ. Ten zbiór jest językiem regularnym nad alfabetem Σ — oznaczanym przez nas jako Q.

Definiujemy język dokumentu JSON (oznaczany przez D) jako zbiór wszystkich ścieżek (słów) od korzenia do dowolnego wierzchołka w drzewie reprezentującym dokument.

Wykonanie zapytania JSONPath na dokumencie JSON można więc zinterpretować jako znalezienie wszystkich słów, z języka D, które należą do języka Q. Innymi słowy — wykonując zapytanie JSONPath najpierw obliczamy przecięcie

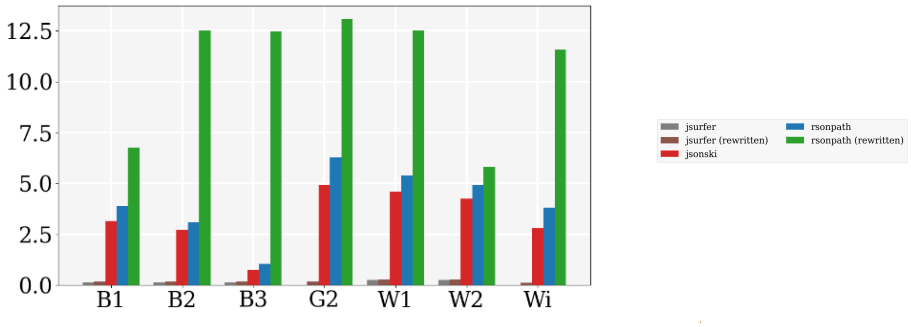


Figure 5: Throughput [GB/s] for descendant-free queries and their rewritings using descendants.

Fig. 2. Porównanie wydajności (ASPLOS '23)

języków D i Q, a następnie wybieramy te wierzchołki w drzewie dokumentu, do których prowadzą ścieżki, które należą do tego języka. Jak opisaliśmy w sekcji 2.2, praktyczne zapytanie wykonywane jest inaczej, ale jest to interpretacja bardzo wygodna do wnioskowania na temat równoważności zapytań.

Ważną cechą jest wybór semantyki dla zapytania. W naszych rozważaniach wszystkie zapytania wykonywane są w *node semantics* — oznacza to, że interesuje nas zbiór elementów z D, niezależnie od tego na ile sposobów możemy daną ścieżkę dopasować do wzorca (w przeciwieństwie do *path semantics*, który jest wrażliwy na to liczbę dopasowania słowa $d \in D$ do wzorca Q).

3 ARCHITEKTURA SYSTEMU

3.1 Ogólna architektura

Wejściem do systemu optymalizatora jest zapytanie JSONPath oraz informacja o strukturze dokumentu, na którym ma zostać ono wykonane. Celem jest wyprodukowanie listy alternatywnych zapytań, które na wejściowym dokumencie zwrócą dokładnie taki sam wynik jak oryginalne zapytanie.

Priorytetem jest poprawność otrzymanych zapytań, dlatego pierwszym zaprojektowanym i zaimplementowanym komponentem systemu został moduł sprawdzania równoważności zapytań 3.2: mając dwa zapytania i dokument, sprawdza czy są one równoważne.

Przy użyciu sprawdzarki równoważności możemy zbudować generator równoważnych zapytań — generujemy „dowolne" zapytania na podstawie oryginału i sprawdzamy ich poprawność, zwracając tylko te równoważne oryginałowi (komponent 3.4). Ponieważ każda taka operacja dzieje się w kontekście jakiegoś dokumentu, ostatnim elementem naszego systemu jest moduł przetwarzający dokument JSON do wygodnej reprezentacji, zawierającej kontekst dla zapytania — komponent 3.3. Architekturę systemu przedstawia rysunek 3.

3.2 Równoważność zapytań

Problem równoważności zapytań stawiamy w następujący sposób: dane są dokument D oraz zapytania Q_1 i Q_2 — zadaniem jest odpowiedź czy zbiór wierzchołków dokumentu D zwrócony w obu przypadkach jest identyczny. Ponieważ definiujemy zapytanie jako funkcję $Z(Q, D) = Q \cap D$, problem równoważności zapytań sprowadza się

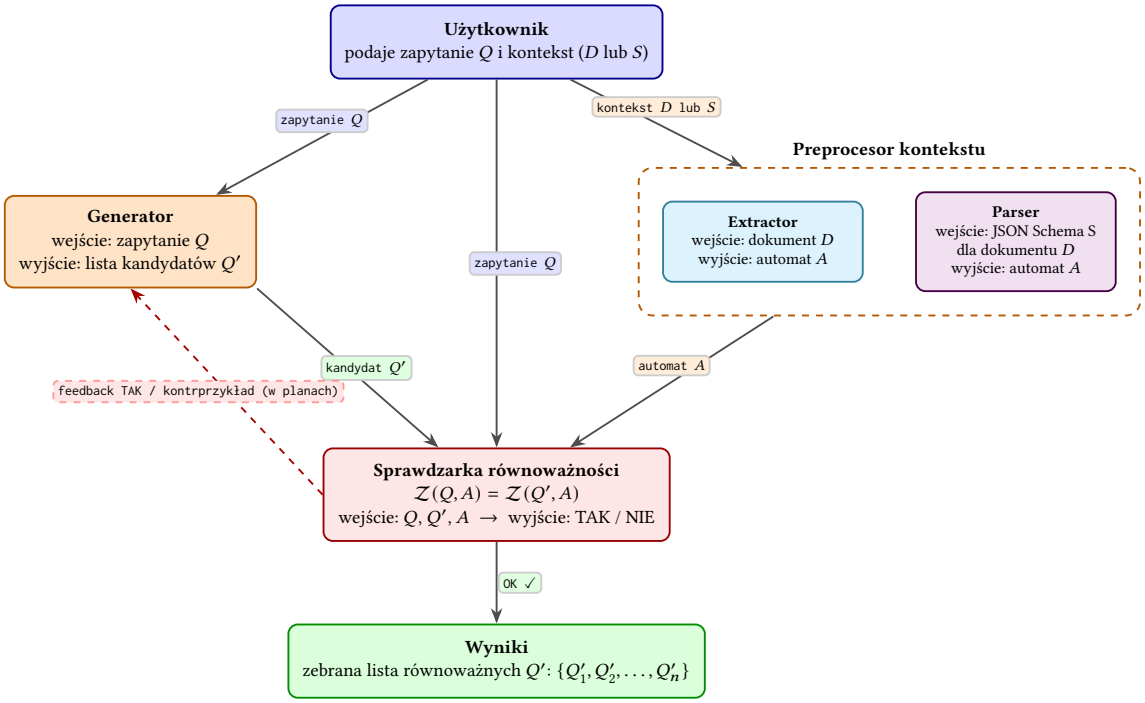


Fig. 3. Ogólna architektura systemu

do porównania $Z(Q_1, D) = Z(Q_2, D)$. Do porównania zapytań Q_1, Q_2 w kontekście dokumentu D wystarczy nam porównanie $Q_1 \cap D = Q_2 \cap D$, przekształcone do postaci równoważnej: $(Q_2 \setminus Q_1) \cup (Q_1 \setminus Q_2) \cap D = \emptyset$.

Taką równoważność sprawdzamy szukając kontrprzykładu w automacie skończonym dla języka $Q_1 \times Q_2 \times D$. Automaty dla Q_1 i Q_2 silnik RSONPath konstruuje przy okazji wykonywania zapytania, natomiast sposób uzyskania automatu reprezentujacego dokument D jest elementem tego projektu — dwie różne metody opisujemy w sekcji 3.3. Otrzymane automaty: dla języka Q_1 automat $A_1 = \langle S_1, \Sigma, \delta_1, q_0, F_1 \rangle$ dla języka Q_2 automat $A_2 = \langle S_2, \Sigma, \delta_2, q_0, F_2 \rangle$ dla dokumentu D automat $A_D = \langle S_D, \Sigma, \delta_D, q_0, F_D \rangle$

Akceptujemy słowo wtedy i tylko wtedy, gdy w automacie dla języka $Q_1 \times Q_2 \times D$ prowadzi ono do stanu (q_1, q_2, d), gdzie d jest stanem akceptującym automatu A_D, a dokładnie jeden ze stanów q_1, q_2 jest akceptujący w odpowiednio A_1 albo A_2. Pseudokod przedstawiono w algorytmie 1.

3.3 Ekstrakcja schematu dokumentu

Podobnie jak w relacyjnych bazach danych rozważania na temat wydajności zapytań muszą mieć na uwadze kontekst danych, na których będą wykonane. Optymalizator działa nie tylko na podstawie zapytania, ale używa dodatkowo informacji o relacjach, które mają istotny wpływ na optymalizacje (klucze, schematy, statystyki). To samo zapytanie może zostać zoptymalizowane na różne sposoby w zależności od kontekstu w którym zostaje wykonane. W przypadku JSONPath tym kontekstem jest dokument JSON, potrzebowaliśmy więc wygodnej reprezentacji dokumentu na cele optymalizatora.

Algorytm 1 Sprawdzanie równoważności zapytań w kontekście dokumentu

Input: automaty A_1, A_2, A_D dla języków Q_1, Q_2, D (deterministyczne, pełne)
Output: `true`, jeśli $Z(Q_1, D) = Z(Q_2, D)$; w przeciwnym razie `false`

$visited \leftarrow \{(q_1^0, q_2^0, d^0)\}$ ▷ stany początkowe A_1, A_2, A_D
$queue \leftarrow [(q_1^0, q_2^0, d^0)]$

while !$queue$.EMPTY() **do**
  $(q_1, q_2, d) \leftarrow queue$.FRONT(); $queue$.POP()
  if IsCounterexample(q_1, q_2, d) **then**
    return `false`
  end if
  for $(q_1', q_2', d') \in$ Successors(q_1, q_2, d) **do**
    if $(q_1', q_2', d') \notin visited$ **then**
      $visited \leftarrow visited \cup \{(q_1', q_2', d')\}$
      $queue$.PUSH$\big((q_1', q_2', d')\big)$
    end if
  end for
end while
return `true`

function Successors(q_1, q_2, d)
$S \leftarrow \emptyset$
for each $a \in \Sigma$ **do**
  $q_1' \leftarrow \delta_1(q_1, a)$ ▷ szukamy przejścia dla danej etykiety, jeśli brak to fallback
  $q_2' \leftarrow \delta_2(q_2, a)$
  $d' \leftarrow \delta_D(d, a)$
  $S \leftarrow S \cup \{(q_1', q_2', d')\}$
end for
return S

function IsCounterexample(q_1, q_2, d)
return $(q_1 \in F_1) \oplus (q_2 \in F_2) \wedge (d \in F_D)$

Dokument będziemy interpretować jako zbiór ścieżek etykiet od korzenia do dowolnego wierzchołka. Ponieważ ten zbiór jest skończony, możemy go reprezentować jako automat. Obecnie opracowaliśmy dwie metody konstrukcji takiego automatu na podstawie wejścia od użytkownika.

3.3.1 Ekstrakcja automatu z dokumentu. Pierwsza metoda przetwarza cały dokument wprost. Rekurencyjnie odwiedza kolejne wierzchołki w drzewie konstruując zbiór ścieżek. Zaletą stworzenia zbioru ścieżek jest znacząca kompresja reprezentacji dokumentu — po pierwsze pomijamy (potencjalnie) duże wartości w liściach, po drugie ścieżki zawierające przejścia przez listę możemy trzymać raz, zamiast trzymania wszystkich elementów listy osobno.

Drzewo JSON jako zbiór wierzchołków (poddokumentów) z krawędziami (etykietami) to w naturalny sposób może być niedeterministyczny automat skończony rozpoznający wszystkie ścieżki. Wybieramy korzeń jako jedyny wierzchołek wejściowy, a wierzchołkami akceptującymi są wszystkie wierzchołki. W takiej reprezentacji występuje bardzo dużo redundancji, której chcemy się pozbyć. Pierwotny pomysł bazuje na idei podobnej do Common Subexpression Elimination [Coc70] — jeśli w naszym automacie istnieją dwa identyczne poddrzewa to możemy zastąpić je jedną kopią, do której prowadzą oba przejścia. Pseudokod algorytmu znajduje się w algorytmie 2.

Algorytm 2 Ekstrakcja automatu z dokumentu

Input: dokument JSON D
Output: automat skończony A rozpoznający wszystkie ścieżki w D

```
States ← ∅                                   ▷ mapa: sygnatura poddrzewa → id stanu
A ← ∅                                        ▷ lista sąsiedztwa stanów automatu
q_0 ← BuildSubtree(root(D), States, A)
mark q_0 as initial state; mark all states as accepting
return A

procedure BuildSubtree(v, States, A)
C ← ∅
for each child c of v do
    h' ← BuildSubtree(c, States, A)
    C ← C ∪ {(c, h')}
end for
h ← HashChildren(C)
if h ∈ States then
    return States[h]
end if
s ← NewState(); States[h] ← s
return s
```

Algorytm ten sprawdza się dobrze przy dokumentach, w których występuje duża ilość identycznych strukturalnie elementów listy. Rozważmy scenariusz, w którym wszystkie elementy tablicy mają ten sam układ pól i różnią się wyłącznie wartościami liści. Po zbudowaniu sygnatur poddrzew można je scalić do jednego wspólnego stanu automatu. Podejście to nie buduje minimalnego automatu, ale jest bardzo solidnym preprocessingiem oczywistych uproszczeń w automacie i może być wejściem do kolejnej fazy — standardowego algorytmu minimalizacji automatu. W naszej implementacji wykorzystaliśmy algorytm Hopcrofta [Hop71]. Co ważne, pierwsza faza pozytywnie wpływa na wydajność algorytmu minimalizacji — w przeciwnym przypadku wejściem do algorytmu Hopcrofta (który jest liniowo logarytmiczny względem rozmiaru wejścia) mógłby być ogromny automat o milionach stanów.

Przykładowo, dokument `walmart_large_record.json` zawiera wejściowo 10 272 545 wierzchołków, a faza preprocessingu zmniejsza go do 5 308 wierzchołków. Warto też dla skali zaznaczyć, że wynikowy, minimalny automat ma 8 stanów. Więcej statystyk związanych z ekstrakcją automatu z dokumentu znajdują się w sekcji 5.2.

W powyższy sposób możemy uzyskać automat dla dokumentu przeglądając jego całą zawartość. Podejście to nakłada większy narzut na optymalizację zapytania niż po prostu wykonanie zapytania na dokumencie w oryginalnej wersji. Zaletą jest jednak to, że mając wiele zapytań do jednego dokumentu, możemy skonstruowany automat wykorzystać wielokrotnie bez potrzeby ponownego przejścia przez cały dokument.

3.3.2 Budowa automatu na podstawie schematu dokumentu. Alternatywnym podejściem byłoby znalezienie formatu pozwalającego na przekazanie podobnej informacji na temat struktury dokumentu w skondensowanej formie. Dla naszego zastosowania potrzebna jest dość restrykcyjna reprezentacja — każda potencjalnie istniejąca w dokumencie ścieżka musi mieć odwzorowanie, w przeciwnym przypadku moglibyśmy błędnie ocenić równoważność zapytań. W naszym projekcie pomysłem jest wprowadzenie typowania dokumentów. Każdy poddokument ma przypisany nazwany typ, który jednoznacznie określa jego strukturę.

Definicja typu składa się z nazwy typu oraz listy par (etykieta, typ), który opisuje jakie etykiety mogą wystąpić w dokumencie danego typu oraz jakiego typu są poddokumenty do których prowadzą poszczególne etykiety. Poniżej przedstawiamy przykład definicji typów:

```
Root:       { content -> ContentList        }
ContentList: { [*]    -> Book | Movie       }
Book:       { title   -> Leaf,
              author  -> Person,
              length  -> Leaf               }
Movie:      { title   -> Leaf,
              cast    -> CastList,
              length  -> Leaf               }
CastList:   { [*]     -> Person             }
Person:     { name    -> Leaf, age -> Leaf  }
Leaf:       {}
```

Powyższa reprezentacja w naturalny sposób może być przestawiona jako automat skończony. Stanem początkowym jest stan odpowiadający typowi Root, a stanami akceptującymi są wszystkie stany. Przejścia między stanami odpowiadają etykietom w definicji typów.

JSON Schema. JSON Schema [JSO20] jest formatem, który pozwala opisywać strukturę dokumentu JSON. W naszym projekcie wykorzystujemy ograniczony podzbiór JSON Schema, który pozwala wyrazić interpretację typów. Prototypowa wersja parsera JSON Schema dla naszych potrzeb posługuje się następującymi założeniami:

- każdy typ ma swoją definicję w sekcji `$defs` JSON Schema.
- typy są proste — każda etykieta prowadzi do dokumentu o typie, który jest nazwany i osobno zdefiniowany. (Planowane jest rozszerzenie tej funkcjonalności o alternatywę typów pod konkretną etykietą.)
- wyjątkiem od powyższego warunku są listy — tam alternatywa jest już wspierana i jeśli etykieta wskazuje na listę, to tam wspierane są heterogeniczne elementy listy.
- ciało dokumentu składa się jedynie z deklaracji korzenia, reszta wynika z zasad umieszczonych w definicjach typów.
- właściwości, które nie są wspierane przez parser, a występują w schemacie są ignorowane.

Przykładowa JSON Schema zawierająca wszystkie akceptowane konstrukcje obecnie w naszym parserze znajduje się w dodatku A.3.

3.4 Generator kandydatów

Mamy do dyspozycji (tani) algorytm, pozwalający określić czy dwa zapytania są równoważne w kontekście danego dokumentu. Do generowania kandydatów dla naszego optymalizatora możemy więc podejść nieostrożnie i odważnie — dowolna próba optymalizatora może być zweryfikowana przed dołączeniem jej do wyniku. Wyzwaniem jest więc „wymyślenie” jak największej ilości kandydatów którzy mogą mieć jakiś sens.

W naszej prototypowej implementacji, wychodząc z pierwotnej inspiracji dla projektu, napisaliśmy jeden podstawowy generator kandydatów na zapytania. Na podstawie zapytania oryginalnego, generowaliśmy kolejne zapytania poprzez

zamianę kolejnych prefiksów zapytania na operator `..` (descendant). Przykładowo, jeśli oryginalne zapytanie to `$.a.b.c`, to kandydatami będą: `$.a.b.c`, `$..b.c` oraz `$..c`.

Ta część jest naszym zdaniem najbardziej podatna na zaawansowane rozszerzenia. Kilka pomysłów na kierunek rozwoju generatora kandydatów podajemy w sekcji 6.1.

4 IMPLEMENTACJA

4.1 Status prac

Implementacja jest obecnie prowadzona jako odgałęzienie oryginalnego repozytorium Git [Git26]. Celem podczas implementacji było ścisłe zintegrowanie z głównym kodem projektu — prowadzenie podobnej struktury modułów, wykorzystywanie struktur i algorytmów, które już były gotowe i są wspólne dla obu części przetwarzania zapytania. W związku z tym w naturalny sposób dziedziczy wykorzystywane technologie i narzędzia. Przede wszystkim Rust jako główny język programowania w implementacji, a wraz z nim cały ekosystem narzędzi takich jak Just, Cargo, Criterion.

Stan implementacji określamy jako *minimum viable product* (MVP). Przygotowane i opisane są interfejsy publiczne każdej z części systemu, z naciskiem na wymienność i generyczność komponentów.

4.2 Szczegóły implementacyjne

Wejściowe zapytanie i dokument przechodzą przez kolejne kroki procesu optymalizacji. `QueryRewritePipeline` jest główną strukturą, która jest inicjalizowana listą obiektów typu `QueryCandidateGenerator`, które powinny zostać użyte.

Wywołanie `QueryRewritePipeline::optimize` powoduje uruchomienie generatorów zapytań umieszczonych w tej strukturze, a każdy kandydat (w formacie zapytania JSONPath) jest kompilowany do automatu RSONPath poprzez istniejący już w RSONPath kompilator.

Dokument lub jego schemat (w zależności od podanego parametru przy uruchomieniu) są przetwarzane do postaci automatu. Zwracanym wynikiem jest lista tych kandydatów (jako zapytań JSONPath), dla których sprawdzenie równoważności wobec dokumentu zakończyło się poprawnie.

4.2.1 Generator kandydatów. Generator kandydatów jest implementowany jako obiekt, który ma udostępniać funkcję (`generate`), która na podstawie oryginalnego zapytania generuje kolejne zapytania. Następnie każde takie zapytanie jest kompilowane.

4.2.2 Równoważność zapytań. Wykorzystując istniejącą już w RSONPath implementację automatu (struktura `Automaton`) napisaliśmy BFS-a, który przechodzi po produkcie kartezjańskim trzech automatów — dwóch zapytań i dokumentu, w którym taka procedura znajdzie kontrprzykład dla równoważności lub zakończy się zwracając prawdę.

4.2.3 Parser JSON Schema. JSON Schema jest parsowany do listy typów rekurencyjnie. Kod parsera wykorzystuje bibliotekę `serde` do rozpakowywania treści JSON do struktury w Rust. Każda definicja typu przetwarzana jest w poszukiwaniu wspieranych słów kluczowych: `properties`, `name`, `type`, `additionalProperties`. Jeśli w definicji typu występuje lista, to tworzony jest nowy, listowy typ, dodawany do listy typów i wynikowego automatu. Przykładowy schemat zawierający wspierane konstrukcje znajduje się w dodatku A.3.

4.2.4 Ekstrakcja schematu dokumentu. W ekstrakcji schematu bezpośrednio z dokumentu również wykorzystujemy bibliotekę `serde` do rozpakowania całego JSON do pamięci, co powinno zostać poprawione w przyszłości. Ta faza,

zgodnie z opisem w sekcji teoretycznej przechodzi po drzewie, utrzymując mapę hashy struktur poddokumentów, które zostały już przetworzone. Obecnie hashowanie realizowane jest prostą strukturą `DefaultHasher` z biblioteki standardowej, ale jest to w prosty sposób wymienne w przyszłości.

4.2.5 Zmiany w implementacji RSONPath. Oryginalna struktura `Automaton` była przewidziana dla reprezentowania automatu związanego z zapytaniem, który ma niewielką ilość stanów, stąd w strukturze stanu nr indeksu był typu `u8` — 256 stanów było zawsze wystarczające, a oszczędność pamięci bardzo istotna dla wydajności całego silnika. W naszej implementacji ta sama struktura używana jest do reprezentowania automatu związanego z dokumentem, który wejściowo ma dużo większą ilość stanów. W implementacji obecnie struktura stanu została zmieniona na typ `u32` — docelowo do napisania jako typ generyczny, który w zależności od kontekstu będzie mógł mieć odpowiedni rozmiar.

5 WYNIKI

5.1 Przepisywanie zapytań i ich wydajność

W naszym projekcie odtworzyliśmy ewaluację zapytań, które pojawiły się w pracy z konferencji ASPLOS '23 [GMP24]. Dla każdego z nich uruchomiliśmy nasz pipeline przepisywania i porównaliśmy wydajność względem oryginalnego zapytania, a także zapytania przepisanego ręcznie w ww. pracy. Wyniki prezentują się następująco (dokładne tabelki z wynikami znajdują się w dodatku A):

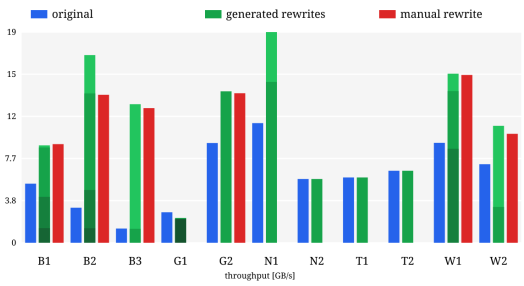


Fig. 4. Wyniki wydajności na architekturze Intel.

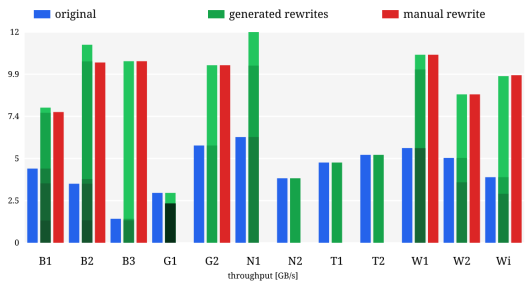


Fig. 5. Wyniki wydajności na architekturze ARM.

5.1.1 Zbiór danych. W celu sprawdzenia działania naszego systemu posługiwaliśmy się tym samym zestawem danych co praca, która była inspiracją dla projektu. Testowane zapytania to te wymienione w pierwszej kolumnie tabeli 1, a dokumenty JSON dostępne są w repozytorium Zenodo [Zen23].

5.1.2 Środowisko eksperymentów. Eksperymenty przeprowadziliśmy na dwóch osobistych maszynach:

- **Intel** — Intel Core i5-1235U, 12 rdzeni, 16 GB RAM, Linux 6.17.0-35-generic (Ubuntu 24.04.4 LTS), z Rustem 1.90.0, flagi SIMD: avx512.
- **ARM** — Apple M3 Pro, 11 rdzeni, 18 GB RAM, macOS 26.5.2 (kernel 25.5.0), z Rustem 1.94.1, flagi SIMD: neon (AdvSIMD).

5.1.3 Sukces. Patrząc na wyniki eksperymentów możemy zaobserwować, że zapytania z benchmarku mają wiele równoważnych odpowiedników. JSONy często mają bardzo regularną strukturę, w której dana etykieta występuje unikalnie na jakiejś głębokości drzewa lub jest dostępna tylko wewnątrz kolejnych poddokumentów. Dzięki temu

Tabela 2. Statystyki ekstrakcji automatu z dokumentu

Dokument	Głębokość	Stanów wejściowych	Po preprocessingu	Min. DFA	Preprocessing [ms]	Hopcroft [ms]
bestbuy	8	42 679 916	424	35	2778.90	162.17
nspl	10	87 746 366	41	28	2988.94	0.79
twitter	12	29 022 795	1 390	106	2140.11	240.40
walmart	5	10 272 545	5 308	8	696.41	500.17
wiki	13	58 815 823	404 627	1 388	32 384.29	268 237.61

można się spodziewać, że wiele dokumentów będzie miało podobny potencjał, który można wykorzystać naszym rozwiązaniem.

Nasz system na podanych przykładach znalazł wiele odpowiedników, w szczególności kilka „agresywniejszych", które ograniczyły zapytania do jednego selektora, będąc w dalszym ciągu równoważne. Z powyższej próbki wynika tendencja, że im krótsze zapytanie w sensie liczby selektorów, tym większe przyspieszenie. Drugim wyróżniającym się czynnikiem jest usunięcie z zapytania wildcardów.

Warto też zauważyć, że wiele zapytań w naszych wynikach osiąga przepustowość mniejszą niż oryginalne zapytania descendant-free. Największe utraty wydajności dzieją się w przypadku, w którym pierwszym selektorem jest `..*`. Wówczas nie uruchamia się head skipping, ponieważ pasuje do niego już pierwszy element, a następnie silnik musi przetworzyć cały dokument, podczas gdy oryginalny, pierwszy selektor *label* mógł już redukować dokument do jednego z poddrzew.

5.1.4 Różnice między Intelem a ARMem. Pracując na maszynach z różnymi architekturami CPU zauważyliśmy różnice w względnej wydajności przepisanych zapytań względem oryginalnych. Różnice jakościowe w wspieraniu SIMD między architekturami x86, a ARM są widoczne w wynikach eksperymentów. x86 ma większe rejestry wektorowe, zatem osiąga większe przyspieszenie dzięki RSONPath i czerpie większe korzyści z przepisanego zapytania niż ta sama optymalizacja na procesorze ARM.

5.2 Ekstrakcja automatu z dokumentu

Czas działania komponentu tworzącego automat dla dokumentu jest istotną częścią czasu działania całego modułu. Tabela 2 przedstawia statystyki ekstrakcji automatu z dokumentów. Wyniki świadczą o wartości prostego preprocessingu, który znacząco zmniejsza liczbę stanów automatu, pozwalając na dalsze operacje. Budowa automatu od razu, bez preprocessingu, mogłaby skończyć się niepowodzeniem z powodu braku pamięci dla tak ogromnych dokumentów. Pomiary wykonano na maszynie Intel opisanej w sekcji 5.1.2.

6 PODSUMOWANIE

W projekcie osiągnęliśmy cel — skonstruowanie modułu do przepisywania zapytań JSONPath w celu poprawienia ich wydajności. Prototypowa implementacja, która jest efektem naszych prac, pozwala na przepisanie zapytań analizowanych w oryginalnej pracy z lepszym efektem niż ich ręczne przepisanie.

Dodatkowo, architektura naszego systemu pozwala na łatwe rozbudowanie w przyszłości — w szczególności na wprowadzenie bardziej zaawansowanych algorytmów przepisywania. Przy ich wprowadzaniu będzie można skorzystać z pozostałych komponentów w celu sprawdzenia poprawności.

6.1 Plany dalszej pracy

Powyższy system jest minimalnym prototypem realizującym ten pomysł. Podczas prac nad implementacją (oraz tą pracą) zauważyliśmy kolejne pola, które w przyszłości mogą zostać ulepszone i rozbudowane:

- Eleganckie rozwiązanie zawiłości implementacyjnych związanych z integrowaniem do RSONPath (typ generyczny w `Automaton`).
- Poszerzenie podzbioru JSON Schema, który jest obsługiwany przez parser.
- Poszerzenie obsługiwanego podzbioru JSONPath przy sprawdzaniu równoważności.
- Optymalizacja wydajności komponentu odpowiedzialnego za ekstrakcję automatu z wejściowego dokumentu — wykorzystanie do tego celu iteratora RSONPath.
- Wprowadzenie bardziej zaawansowanych algorytmów przepisywania zapytań JSONPath.
- Eksperymenty z dużymi modelami językowymi jako generatorami kandydatów na zapytanie.
- Wprowadzenie większej ilości zbiorów danych i typów zapytań do eksperymentowania w celu znalezienia najlepszych heurystyk dla przepisywania.

A WYNIKI EKSPERYMENTÓW

Pełne wyniki pomiarów przedstawiamy w tabelach 3 i 4. Kolumna *Speedup* oznacza przyspieszenie względem wariantu `paper_original` dla tego samego zapytania i dokumentu. Źródłowe pliki CSV znajdują się w katalogu `data/`.

A.1 Wyniki na architekturze Intel

Tabela 3. Szczegółowe wyniki benchmarków na architekturze Intel

ID	Dataset	Wariant	Ranga	Zapytanie	# wyn.	Czas [ms]	Speedup
B1	bestbuy	oryginał	—	`$['products'][*]['categoryPath'][*]['id']`	697 440	193.7	1.00
B1	bestbuy	auto (najlepsze)	—	`$..['categoryPath'][*]['id']`	697 440	117.5	1.65
B1	bestbuy	auto (kandydat)	1	`$..['categoryPath'][*]['id']`	697 440	117.5	1.65
B1	bestbuy	auto (kandydat)	2	`$..['id']`	697 440	120.1	1.61
B1	bestbuy	auto (kandydat)	3	`$..[*]['categoryPath'][*]['id']`	697 440	249.3	0.78
B1	bestbuy	auto (kandydat)	4	`$..[*]['id']`	697 440	784.5	0.25
B1	bestbuy	ręczne	—	`$..['categoryPath']..['id']`	697 440	116.1	1.67
B2	bestbuy	oryginał	—	`$['products'][*]['videoChapters'][*]['chapter']`	8 857	326.6	1.00
B2	bestbuy	auto (najlepsze)	—	`$..['videoChapters'][*]['chapter']`	8 857	76.7	4.26
B2	bestbuy	auto (kandydat)	1	`$..['chapter']`	8 852	61.0	5.35
B2	bestbuy	auto (kandydat)	2	`$..['videoChapters'][*]['chapter']`	8 857	76.7	4.26
B2	bestbuy	auto (kandydat)	3	`$..[*]['videoChapters'][*]['chapter']`	8 857	217.0	1.50
B2	bestbuy	auto (kandydat)	4	`$..[*]['chapter']`	8 857	789.7	0.41
B2	bestbuy	ręczne	—	`$..['videoChapters']..['chapter']`	8 857	77.4	4.22
B3	bestbuy	oryginał	—	`$['products'][*]['videoChapters']`	769	804.7	1.00
B3	bestbuy	auto (najlepsze)	—	`$..['videoChapters']`	769	82.6	9.74
B3	bestbuy	auto (kandydat)	1	`$..['videoChapters']`	769	82.6	9.74
B3	bestbuy	auto (kandydat)	2	`$..[*]['videoChapters']`	769	820.8	0.98
B3	bestbuy	ręczne	—	`$..['videoChapters']`	769	85.0	9.46
G1	google_map	oryginał	—	`$[*]['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	408.1	1.00
G1	google_map	auto (najlepsze)	—	`$[*]['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	408.1	1.00
G1	google_map	auto (kandydat)	1	`$..[*]['steps'][*]['distance']['text']`	1 716 752	501.4	0.81
G1	google_map	auto (kandydat)	2	`$..['steps'][*]['distance']['text']`	1 716 752	503.0	0.81
G1	google_map	auto (kandydat)	3	`$..['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	519.3	0.79
G1	google_map	auto (kandydat)	4	`$..[*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	522.9	0.78

Kontynuacja na następnej stronie

Tabela 3. (cd.) Szczegółowe wyniki benchmarków na architekturze Intel

ID	Dataset	Wariant	Ranga	Zapytanie	# wyn.	Czas [ms]	Speedup
G1	google_map	auto (kandydat)	5	`$..['legs'][*]['steps'][*]['distance']['text']`	1 716 752	533.8	0.76
G2	google_map	oryginał	—	`$[*]['available_travel_modes']`	90	124.8	1.00
G2	google_map	auto (najlepsze)	—	`$..['available_travel_modes']`	90	82.2	1.52
G2	google_map	auto (kandydat)	1	`$..['available_travel_modes']`	90	82.2	1.52
G2	google_map	ręczne	—	`$..['available_travel_modes']`	90	83.3	1.50
N1	nspl	oryginał	—	`$['meta']['view']['columns'][*]['name']`	44	110.9	1.00
N1	nspl	auto (najlepsze)	—	`$..['view']['columns'][*]['name']`	44	63.0	1.76
N1	nspl	auto (kandydat)	1	`$..['view']['columns'][*]['name']`	44	63.0	1.76
N1	nspl	auto (kandydat)	2	`$..['columns'][*]['name']`	44	82.5	1.34
N2	nspl	oryginał	—	`$['data'][*][*][*]`	8 774 410	208.0	1.00
N2	nspl	auto (najlepsze)	—	`$['data'][*][*][*]`	8 774 410	208.0	1.00
T1	twitter	oryginał	—	`$[*]['entities']['urls'][*]['url']`	88 881	141.4	1.00
T1	twitter	auto (najlepsze)	—	`$[*]['entities']['urls'][*]['url']`	88 881	141.4	1.00
T2	twitter	oryginał	—	`$[*]['text']`	150 135	128.2	1.00
T2	twitter	auto (najlepsze)	—	`$[*]['text']`	150 135	128.2	1.00
W1	walmart	oryginał	—	`$['items'][*]['bestMarketplacePrice']['price']`	15 892	109.2	1.00
W1	walmart	auto (najlepsze)	—	`$..['bestMarketplacePrice']['price']`	15 892	64.5	1.69
W1	walmart	auto (kandydat)	1	`$..['bestMarketplacePrice']['price']`	15 892	64.5	1.69
W1	walmart	auto (kandydat)	2	`$..['price']`	15 892	71.9	1.52
W1	walmart	auto (kandydat)	3	`$..[*]['bestMarketplacePrice']['price']`	15 892	116.0	0.94
W1	walmart	ręczne	—	`$..['bestMarketplacePrice']['price']`	15 892	65.0	1.68
W2	walmart	oryginał	—	`$['items'][*]['name']`	272 499	138.9	1.00
W2	walmart	auto (najlepsze)	—	`$..['name']`	272 499	93.3	1.49
W2	walmart	auto (kandydat)	1	`$..['name']`	272 499	93.3	1.49
W2	walmart	auto (kandydat)	2	`$..[*]['name']`	272 499	304.5	0.46
W2	walmart	ręczne	—	`$..['name']`	272 499	100.1	1.39

A.2 Wyniki na architekturze ARM

Tabela 4. Szczegółowe wyniki benchmarków na architekturze ARM

ID	Dataset	Wariant	Ranga	Zapytanie	# wyn.	Czas [ms]	Speedup
B1	bestbuy	oryginał	—	`$['products'][*]['categoryPath'][*]['id']`	697 440	239.2	1.00
B1	bestbuy	auto (kandydat)	1	`$['products'][*]['categoryPath'][*]['id']`	697 440	239.2	1.00
B1	bestbuy	auto (kandydat)	2	`$..['id']`	697 440	131.3	1.82
B1	bestbuy	auto (kandydat)	3	`$..['categoryPath'][*]['id']`	697 440	136.5	1.75
B1	bestbuy	auto (kandydat)	4	`$..[*]['categoryPath'][*]['id']`	697 440	298.7	0.80
B1	bestbuy	auto (kandydat)	5	`$..[*]['id']`	697 440	798.6	0.30
B1	bestbuy	ręczne	—	`$..['categoryPath']..['id']`	697 440	135.7	1.76
B2	bestbuy	oryginał	—	`$['products'][*]['videoChapters'][*]['chapter']`	8 857	300.5	1.00
B2	bestbuy	auto (kandydat)	1	`$['products'][*]['videoChapters'][*]['chapter']`	8 857	300.5	1.00
B2	bestbuy	auto (kandydat)	2	`$..['chapter']`	8 851	89.6	3.35
B2	bestbuy	auto (kandydat)	3	`$..['videoChapters'][*]['chapter']`	8 857	97.8	3.07
B2	bestbuy	auto (kandydat)	4	`$..[*]['videoChapters'][*]['chapter']`	8 857	278.6	1.08
B2	bestbuy	auto (kandydat)	5	`$..[*]['chapter']`	8 857	790.8	0.38
B2	bestbuy	ręczne	—	`$..['videoChapters']..['chapter']`	8 857	98.4	3.05
B3	bestbuy	oryginał	—	`$['products'][*]['videoChapters']`	769	739.4	1.00
B3	bestbuy	auto (kandydat)	1	`$['products'][*]['videoChapters']`	769	739.4	1.00
B3	bestbuy	auto (kandydat)	2	`$..['videoChapters']`	769	97.7	7.57
B3	bestbuy	auto (kandydat)	3	`$..[*]['videoChapters']`	769	790.3	0.94
B3	bestbuy	ręczne	—	`$..['videoChapters']`	769	97.7	7.57

Kontynuacja na następnej stronie

Tabela 4. (cd.) Szczegółowe wyniki benchmarków na architekturze ARM

ID	Dataset	Wariant	Ranga	Zapytanie	# wyn.	Czas [ms]	Speedup
G1	google_map	oryginał	—	`$[*]['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	386.8	1.00
G1	google_map	auto (kandydat)	1	`$[*]['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	386.8	1.00
G1	google_map	auto (kandydat)	2	`$..['steps'][*]['distance']['text']`	1 716 752	488.1	0.79
G1	google_map	auto (kandydat)	3	`$..['legs'][*]['steps'][*]['distance']['text']`	1 716 752	488.9	0.79
G1	google_map	auto (kandydat)	4	`$..[*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	490.1	0.79
G1	google_map	auto (kandydat)	5	`$..['routes'][*]['legs'][*]['steps'][*]['distance']['text']`	1 716 752	490.5	0.79
G1	google_map	auto (kandydat)	6	`$..[*]['steps'][*]['distance']['text']`	1 716 752	491.9	0.79
G2	google_map	oryginał	—	`$[*]['available_travel_modes']`	90	198.4	1.00
G2	google_map	auto (kandydat)	1	`$[*]['available_travel_modes']`	90	198.4	1.00
G2	google_map	auto (kandydat)	2	`$..['available_travel_modes']`	90	108.7	1.83
G2	google_map	ręczne	—	`$..['available_travel_modes']`	90	108.7	1.83
N1	nspl	oryginał	—	`$['meta']['view']['columns'][*]['name']`	44	194.4	1.00
N1	nspl	auto (kandydat)	1	`$['meta']['view']['columns'][*]['name']`	44	194.4	1.00
N1	nspl	auto (kandydat)	2	`$..['view']['columns'][*]['name']`	44	97.5	1.99
N1	nspl	auto (kandydat)	3	`$..['columns'][*]['name']`	44	116.0	1.68
N2	nspl	oryginał	—	`$['data'][*][*][*]`	8 774 410	318.6	1.00
N2	nspl	auto (kandydat)	1	`$['data'][*][*][*]`	8 774 410	318.6	1.00
T1	twitter	oryginał	—	`$[*]['entities']['urls'][*]['url']`	88 881	178.4	1.00
T1	twitter	auto (kandydat)	1	`$[*]['entities']['urls'][*]['url']`	88 881	178.4	1.00
T2	twitter	oryginał	—	`$[*]['text']`	150 135	162.8	1.00
T2	twitter	auto (kandydat)	1	`$[*]['text']`	150 135	162.8	1.00
W1	walmart	oryginał	—	`$['items'][*]['bestMarketplacePrice']['price']`	15 892	178.5	1.00
W1	walmart	auto (kandydat)	1	`$['items'][*]['bestMarketplacePrice']['price']`	15 892	178.5	1.00
W1	walmart	auto (kandydat)	2	`$..['bestMarketplacePrice']['price']`	15 892	89.9	1.99
W1	walmart	auto (kandydat)	3	`$..['price']`	15 892	97.5	1.83
W1	walmart	auto (kandydat)	4	`$..[*]['bestMarketplacePrice']['price']`	15 892	178.9	1.00
W1	walmart	ręczne	—	`$..['bestMarketplacePrice']['price']`	15 892	89.9	1.99
W2	walmart	oryginał	—	`$['items'][*]['name']`	272 499	199.2	1.00
W2	walmart	auto (kandydat)	1	`$['items'][*]['name']`	272 499	199.2	1.00
W2	walmart	auto (kandydat)	2	`$..['name']`	272 499	113.9	1.75
W2	walmart	auto (kandydat)	3	`$..[*]['name']`	272 499	279.5	0.71
W2	walmart	ręczne	—	`$..['name']`	272 499	113.9	1.75
Wi	wikidata	oryginał	—	`$[*]['claims']['P150'][*]['mainsnak']['property']`	15 603	284.6	1.00
Wi	wikidata	auto (kandydat)	1	`$[*]['claims']['P150'][*]['mainsnak']['property']`	15 603	284.6	1.00
Wi	wikidata	auto (kandydat)	2	`$..['P150'][*]['mainsnak']['property']`	15 603	112.0	2.54
Wi	wikidata	auto (kandydat)	3	`$..['claims']['P150'][*]['mainsnak']['property']`	15 603	381.1	0.75
Wi	wikidata	ręczne	—	`$..['P150']..['mainsnak']['property']`	15 603	111.4	2.56

A.3 Przykładowy dokument JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$defs": {
    "Root": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "content": {
          "type": "array",
          "items": {
            "anyOf": [{ "$ref": "#/$defs/Book" }, { "$ref": "#/$defs/Movie" }]
          }
        }
      }
    },
    "Book": {
```

```
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "title": { "type": "string" },
        "author": { "$ref": "#/$defs/Person" },
        "length": { "type": "integer"}
      }
    },
    "Movie": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "title": { "type": "string" },
        "cast": { "type": "array", "items": { "$ref": "#/$defs/Person" } },
        "length": { "type": "integer"}
      }
    },
    "Person": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "name": { "type": "string" },
        "age": { "type": "integer"}
      }
    }
  },
  "$ref": "#/$defs/Root"
}
```

LITERATURA

[Coc70] John Cocke. Global common subexpression elimination. In *Proceedings of a Symposium on Compiler Optimization*, pages 20–24, New York, NY, USA, 1970. Association for Computing Machinery.

[Git26] 2026. GitHub fork of RsonPath.

[GMP24] Mateusz Gienieczko, Filip Murlak, and Charles Paperman. Supporting descendants in simd-accelerated jsonpath. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '23, page 338–361, New York, NY, USA, 2024. Association for Computing Machinery.

[GNB24] Stefan Gössner, Glyn Normington, and Carsten Bormann. JSONPath: Query Expressions for JSON. RFC 9535, February 2024.

[Hop71] John Hopcroft. An n log n algorithm for minimizing states in a finite automaton. In Zvi Kohavi and Azaria Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[JSO20] 2020. Definition of JSON Schema, a vocabulary that allows you to annotate and validate JSON documents.

[Lam75] Leslie Lamport. Multiple byte processing with full-word instructions. *Communications of the ACM*, 18(8):471–475, August 1975.

[Pra22] 2022.

[RSO26] Dostęp: 25.06.2026. Repozytorium projektu RSONPath na GitHubie.

[Zen23] Dataset for bencharmking rsonpath, 2023. Charles Paperman, Mateusz Gienieczko, Filip Murlak. Version v2. Zbiór dokumentów JSON i zapytań użytych w ewaluacji z pracy ASPLOS'23.